Top 50 SQL Interview Questions and Answers with Examples

1. What is SQL?

SQL (Structured Query Language) is used to manage and manipulate relational databases. It allows users to **create**, **read**, **update**, and **delete** data (CRUD operations). SQL is used by popular databases like **MySQL, PostgreSQL, Oracle, and SQL Server**. It provides commands like SELECT, INSERT, UPDATE, and DELETE. SQL follows the **ACID properties** to ensure data integrity.

2. Types of SQL Commands?

- **DDL (Data Definition Language)** → Commands that define the structure of the database (CREATE, ALTER, DROP).
- **DML (Data Manipulation Language)** → Commands that manipulate data (INSERT, UPDATE, DELETE).
- **DCL (Data Control Language)** → Commands that manage access (GRANT, REVOKE).
- **TCL (Transaction Control Language)** → Commands that manage transactions (COMMIT, ROLLBACK, SAVEPOINT).

3. Difference between DDL, DML, DCL, TCL?

DDL modifies database structure, while DML modifies data. DCL manages permissions, while TCL handles transactions.

- **DDL Example**: CREATE TABLE Employees (EmpID INT PRIMARY KEY, Name VARCHAR(50));
- **DML Example**: INSERT INTO Employees VALUES (1, 'John');
- **DCL Example**: GRANT SELECT ON Employees TO User1;
- **TCL Example**: COMMIT; to save all changes.

4. What is a Primary Key?

A **Primary Key** uniquely identifies each row in a table. It **cannot be NULL** and must be **unique**.

```
CREATE TABLE Employees (
    EmpID INT PRIMARY KEY,
```

```
    Name VARCHAR(50)
);
```

Here, EmpID is the **Primary Key**, ensuring unique identification of employees.

5. What is a Foreign Key?

A **Foreign Key** establishes a relationship between two tables by linking a column to the **Primary Key** of another table.

```
CREATE TABLE Orders (
    OrderID INT PRIMARY KEY,
    EmpID INT,
    FOREIGN KEY (EmpID) REFERENCES Employees(EmpID)
);
```

This ensures that EmpID in Orders must exist in Employees.

6. What is a Unique Key?

A **Unique Key** ensures that column values remain unique but **allows NULL** values.

```
CREATE TABLE Users (
    Email VARCHAR(100) UNIQUE
);
```

Unlike **Primary Key**, multiple **Unique Keys** can exist in a table.

7. Difference between Primary Key and Unique Key?

Feature	**Primary Key**	**Unique Key**
Uniqueness	Ensures unique values	Ensures unique values
NULL Allowed?	No	Yes (one or more)
Number per Table	Only one	Multiple allowed

8. What is a Candidate Key?

A **Candidate Key** is a column (or set of columns) that could be a **Primary Key**.

- If a table has EmpID and Email as unique, both are **Candidate Keys**, but only one can be the **Primary Key**.

9. Difference between DELETE, TRUNCATE, DROP?

- **DELETE** → Removes specific rows, can be rolled back (DELETE FROM Employees WHERE EmpID = 1;).
- **TRUNCATE** → Removes all rows, cannot be rolled back (TRUNCATE TABLE Employees;).
- **DROP** → Deletes entire table including structure (DROP TABLE Employees;).

10. What are Constraints?

Constraints ensure **data integrity** in SQL tables. Common constraints include:

- NOT NULL → Prevents NULL values.
- UNIQUE → Ensures uniqueness.
- PRIMARY KEY → Combines NOT NULL + UNIQUE.
- CHECK → Enforces conditions (Salary > 30000).
- DEFAULT → Assigns default values (Gender CHAR(1) DEFAULT 'M').

---

11. What is Normalization?

Normalization organizes data efficiently by reducing redundancy.

- **1NF:** Atomic values (no multiple values in a single column).
- **2NF:** 1NF + No **Partial Dependency** (non-key columns depend on primary key).
- **3NF:** 2NF + No **Transitive Dependency** (non-key columns don't depend on each other).
- **BCNF:** Stronger form of **3NF**, ensuring every determinant is a **Candidate Key**.

12. What is Denormalization?

Denormalization adds redundancy for performance improvement.

- Instead of using **JOINs**, data is stored redundantly to reduce complex queries.

13. What is an Index?

An **Index** speeds up queries by improving search performance.

CREATE INDEX idx_name ON Employees(Name);

Indexes **reduce read time** but increase write time due to extra maintenance.

14. Clustered vs Non-Clustered Index?

Clustered Index	**Non-Clustered Index**
Sorts table data	Uses a separate structure
One per table	Multiple allowed
Faster for range queries	Slower for range queries

CREATE CLUSTERED INDEX idx_empid ON Employees(EmpID);

CREATE NONCLUSTERED INDEX idx_name ON Employees(Name);

15. What are Views?

A **View** is a virtual table that displays data from one or more tables.

CREATE VIEW EmployeeView AS

SELECT EmpID, Name FROM Employees;

Views **improve security** by restricting column access.

16. What are Stored Procedures?

Stored Procedures are precompiled SQL code that can be reused.

CREATE PROCEDURE GetEmployees()

AS

SELECT * FROM Employees;

They improve performance and **reduce SQL injection risks**.

17. What are Triggers?

Triggers execute **automatically** when an event occurs.

```
CREATE TRIGGER AfterInsert
AFTER INSERT ON Employees
FOR EACH ROW
BEGIN
  INSERT INTO LogTable VALUES (NEW.EmpID, NOW());
END;
```

Useful for **logging, validation, and enforcing business rules**.

18. HAVING vs WHERE?

- **WHERE** → Filters **before** aggregation.
- **HAVING** → Filters **after** aggregation.

```
SELECT Department, COUNT(*) FROM Employees
WHERE Salary > 50000
GROUP BY Department
HAVING COUNT(*) > 5;
```

Filters departments with **more than 5 employees** earning over 50,000.

19. Difference between JOINs?

JOIN Type	Description
INNER JOIN	Only matching rows
LEFT JOIN	All from left + matching right
RIGHT JOIN	All from right + matching left
FULL JOIN	All from both tables

```
SELECT E.Name, O.OrderID FROM Employees E
INNER JOIN Orders O ON E.EmpID = O.EmpID;
```

20. Difference between UNION and UNION ALL?

- **UNION** removes duplicates.

- **UNION ALL** keeps duplicates.

SELECT Name FROM Employees

UNION

SELECT Name FROM Clients;

This returns unique names from both tables. Using UNION ALL keeps duplicate names.

---

21. What is a Self Join?

A **Self Join** joins a table with itself.

SELECT A.Name AS Employee, B.Name AS Manager

FROM Employees A

JOIN Employees B ON A.ManagerID = B.EmpID;

This fetches employees and their **managers** from the same table.

---

22. What is a Cross Join?

A **Cross Join** creates a **Cartesian product** of two tables.

SELECT E.Name, P.ProjectName

FROM Employees E

CROSS JOIN Projects P;

Each employee gets **paired** with every project.

---

23. What is a Common Table Expression (CTE)?

A **CTE** is a temporary result set that improves readability.

```
WITH EmployeeCTE AS (
    SELECT Name, Salary FROM Employees WHERE Salary > 50000
)
SELECT * FROM EmployeeCTE;
```

Useful for **recursive queries**.

---

24. What is a Recursive CTE?

It helps retrieve hierarchical data, like organizational structures.

```
WITH EmployeeHierarchy AS (
   SELECT EmpID, Name, ManagerID FROM Employees WHERE ManagerID IS NULL
   UNION ALL
   SELECT E.EmpID, E.Name, E.ManagerID
   FROM Employees E JOIN EmployeeHierarchy EH ON E.ManagerID = EH.EmpID
)
SELECT * FROM EmployeeHierarchy;
```

This retrieves employees **with their reporting hierarchy**.

---

25. What is a Window Function?

A **Window Function** performs calculations across a subset of rows **without** collapsing them.

```
SELECT Name, Salary,
    RANK() OVER (ORDER BY Salary DESC) AS Rank
FROM Employees;
```

Assigns a **salary rank** to each employee **without grouping**.

---

26. What is a Rank Function?

RANK(), **DENSE_RANK()**, and **ROW_NUMBER()** provide row numbers.

```
SELECT Name, Salary,
    RANK() OVER (ORDER BY Salary DESC) AS Rank,
```

```
    DENSE_RANK() OVER (ORDER BY Salary DESC) AS DenseRank,
    ROW_NUMBER() OVER (ORDER BY Salary DESC) AS RowNum
FROM Employees;
```

- **RANK()** → Skips numbers for ties.
- **DENSE_RANK()** → No gaps for ties.
- **ROW_NUMBER()** → Assigns a unique row number.

---

27. Difference between EXISTS and IN?

- **EXISTS** stops at the first match, better for large datasets.
- **IN** compares all values, better for smaller datasets.

```
SELECT Name FROM Employees WHERE EXISTS (
  SELECT 1 FROM Orders WHERE Employees.EmpID = Orders.EmpID
);
```

Returns employees **who have orders**.

---

28. What is a CASE Statement?

A **CASE** statement is like an IF-ELSE in SQL.

```
SELECT Name, Salary,
    CASE
      WHEN Salary > 80000 THEN 'High'
      WHEN Salary BETWEEN 50000 AND 80000 THEN 'Medium'
      ELSE 'Low'
    END AS SalaryCategory
FROM Employees;
```

Categorizes **employees based on salary**.

---

29. What is COALESCE?

COALESCE returns the first **non-NULL** value.

SELECT Name, COALESCE(Phone, 'No Phone') AS Contact

FROM Employees;

Replaces NULL phone numbers with **"No Phone"**.

---

30. What is NULLIF?

NULLIF(A, B) returns NULL if **A = B**, otherwise it returns **A**.

SELECT NULLIF(10, 10); -- Returns NULL

SELECT NULLIF(10, 5); -- Returns 10

Useful for **avoiding division by zero errors**.

---

31. What is GROUP BY?

It groups results based on a column.

SELECT Department, COUNT(*) FROM Employees

GROUP BY Department;

Counts employees **per department**.

---

32. Difference between COUNT(*), COUNT(1), COUNT(column)?

- **COUNT(*)** counts **all rows** including NULLs.
- **COUNT(1)** is optimized like COUNT(*).
- **COUNT(column)** counts **only non-null values**.

SELECT COUNT(*) FROM Employees; -- Includes NULLs

SELECT COUNT(Salary) FROM Employees; -- Excludes NULL salaries

---

33. What is a Subquery?

A **Subquery** is a query inside another query.

```
SELECT Name FROM Employees WHERE Salary >
   (SELECT AVG(Salary) FROM Employees);
```

Finds employees earning **above average salary**.

---

34. What is a Correlated Subquery?

It references the **outer query** for each row.

```
SELECT Name FROM Employees E1
WHERE Salary > (SELECT AVG(Salary) FROM Employees E2 WHERE
E1.Department = E2.Department);
```

Finds employees **earning above their department's average salary**.

---

35. What is PIVOT?

It transforms **rows into columns**.

```
SELECT * FROM (
   SELECT Department, Gender FROM Employees
) AS SourceTable
PIVOT (
   COUNT(Gender) FOR Gender IN ([Male], [Female])
) AS PivotTable;
```

Counts **Male/Female employees per department**.

---

36. What is an Aggregate Function?

Functions like SUM(), AVG(), MIN(), MAX(), and COUNT().

```
SELECT AVG(Salary) FROM Employees;
```

Finds **average salary**.

---

37. How to find duplicate records?

```
SELECT Name, COUNT(*) FROM Employees
GROUP BY Name HAVING COUNT(*) > 1;
```

Lists **duplicate employee names**.

---

38. How to delete duplicate records?

```
WITH CTE AS (
   SELECT *, ROW_NUMBER() OVER (PARTITION BY Name ORDER BY EmpID) AS RowNum
   FROM Employees
)
DELETE FROM Employees WHERE EmpID IN (
   SELECT EmpID FROM CTE WHERE RowNum > 1
);
```

Keeps only the **first occurrence**.

---

39. What is ACID in SQL?

- **Atomicity** → All operations succeed or none do.
- **Consistency** → Data remains valid before/after transactions.
- **Isolation** → Transactions execute independently.
- **Durability** → Changes persist even after system failure.

---

40. What is Deadlock?

A **deadlock** occurs when two transactions **wait for each other** to release a resource.

```
BEGIN TRANSACTION;
UPDATE Employees SET Salary = Salary + 5000 WHERE EmpID = 1;
```

```
WAITFOR DELAY '00:00:05'; -- Simulates delay
UPDATE Orders SET Amount = Amount - 500 WHERE OrderID = 2;
COMMIT;
```

If another transaction updates **Orders first**, a deadlock happens.

41. What are Indexes in SQL?

Indexes improve **query performance** by making searches faster.

```
CREATE INDEX idx_employee_name ON Employees(Name);
```

This creates an **index on the Name column**, speeding up WHERE Name = 'John' queries.

---

42. What are the types of Indexes in SQL?

- **Clustered Index** → Data is stored **physically** in sorted order (Only one per table).
- **Non-Clustered Index** → Separate structure for searching, pointing to actual rows.

```
CREATE CLUSTERED INDEX idx_emp_salary ON Employees(Salary);
CREATE NONCLUSTERED INDEX idx_emp_dept ON Employees(Department);
```

Clustered **affects data storage**, while non-clustered **only improves search performance**.

---

43. What is a Stored Procedure?

A **stored procedure** is a precompiled SQL query for better performance.

```
CREATE PROCEDURE GetEmployeesByDept (@DeptName VARCHAR(50))
AS
BEGIN
   SELECT * FROM Employees WHERE Department = @DeptName;
END;
```

Call it with:

EXEC GetEmployeesByDept 'IT';

This fetches **all employees in the IT department**.

---

44. What is a Trigger in SQL?

A **trigger** automatically executes on INSERT, UPDATE, or DELETE.

```
CREATE TRIGGER trg_AfterInsert
ON Employees
AFTER INSERT
AS
BEGIN
   PRINT 'A new employee record has been inserted!';
END;
```

Triggers **enforce business rules** automatically.

---

45. What is Partitioning in SQL?

Partitioning **divides large tables** into smaller parts for better performance.

```
CREATE PARTITION FUNCTION pfSalaryRange (INT)
AS RANGE LEFT FOR VALUES (50000, 100000, 150000);
```

This splits **salary ranges** into different partitions.

---

46. What is the Difference Between DELETE, TRUNCATE, and DROP?

- **DELETE** → Removes specific rows, can be rolled back.
- **TRUNCATE** → Removes all rows, **faster** than DELETE, but cannot be rolled back.
- **DROP** → Deletes the entire table.

```
DELETE FROM Employees WHERE EmpID = 101; -- Removes one record
```

TRUNCATE TABLE Employees; -- Removes all records

DROP TABLE Employees; -- Deletes the entire table

---

47. What is the Difference Between HAVING and WHERE?

- **WHERE** filters before aggregation.
- **HAVING** filters after aggregation.

SELECT Department, AVG(Salary)

FROM Employees

WHERE Salary > 50000 -- Filters individual rows

GROUP BY Department

HAVING AVG(Salary) > 60000; -- Filters aggregated results

This returns **departments** where the **average salary is above 60,000**.

---

48. What is SQL Injection and How to Prevent It?

SQL Injection is a **security vulnerability** where attackers inject malicious SQL. Example of vulnerable query:

SELECT * FROM Users WHERE Username = 'admin' AND Password = ' ' OR 1=1 --';

Use **parameterized queries** to prevent it:

SELECT * FROM Users WHERE Username = @Username AND Password = @Password;

Always **validate user inputs**.

---

49. How to Optimize SQL Queries?

1. **Use Indexes** → Speeds up searches.
2. **Avoid SELECT *** → Fetch only required columns.

3. **Use Joins Efficiently** → Prefer INNER JOIN over OUTER JOIN when possible.
4. **Use EXISTS Instead of IN** → Faster in large datasets.
5. **Use LIMIT or TOP** → Fetch only required rows.

Example of optimization:

SELECT Name, Salary FROM Employees WHERE Salary > 80000;

Fetching **only needed columns** reduces load.

---

50. What is the Difference Between OLTP and OLAP?

- **OLTP (Online Transaction Processing)** → Handles fast, real-time transactions (e.g., **banking systems**).
- **OLAP (Online Analytical Processing)** → Handles complex reporting and analytics (e.g., **business intelligence**).

Feature	**OLTP**	**OLAP**
Purpose	Transactions	Reporting
Queries	Simple & Fast	Complex
Data Volume	Small	Large
Example	ATM System	Data Warehouse